

Parallel Programming

Lecture Notes · Script

ETH Zürich · 2nd Semester · Part 1 & Part 2

Contents

Part 1

- 01 EProg Recap
- 02 Processes and Threads
- 03 Thread Join
- 04 Shared Resources
- 05 Hardware
- 06 Pipelining
- 07 Scalability
- 08 Arbeitsaufteilung
- 09 Task graphs
- 10 External Locks, Race Conditions
- 11 Guidelines

Part 2

- 01 Hardware Revamped
- 02 State Space Diagram
- 03 Software Locks
- 04 Hardware Locks
- 05 Deadlocks
- 06 Semaphores
- 07 Barrier
- 08 Producer Consumer
- 09 Monitors
- 10 Reader Writer Locks
- 11 Lock Granularity
- 12 Lock Free
- 13 Execution Timelines
- 14 Consensus
- 15 Transactional Memory

Part 1

Foundations · Threads · Hardware · Scalability

01 EProg Recap

When an object is passed as a parameter, the object is not copied. The parameter refers to the same object. If the parameter is modified, it will affect the original object.

Static variables are shared by all the instances of the class.

Static constants are final variables shared by all the instances of the class.

Class methods are marked by the static keyword. Class methods are not called via an object reference but directly via the class. Can be called from a static context (e.g., main method). Class methods often serve as utility functions. They're generic and need no access to object variables and methods.

02 Processes and Threads

Parallel output order changes between runs, because **console access is random**.

Single-Core

- A single-core CPU can execute one instruction at a time, meaning sequentially.
- processes share CPU, but **each process has own memory space**
- während wir auf Nutzereingabe warten, ist CPU auf **idle**, was man vermeiden möchte
- die OS scheduler sind für die Arbeitsverwaltung zuständig
- der **CPU scheduler** verteilt CPU zwischen Prozessen (zwischen Prozessen wechseln)

Every process needs context (PCB)

- **hardware** context
- **memory** context
- **os-level** context
- changing to another process requires a context switch, resulting in large overhead
- swap memory (green) is a lot slower

OS manages processes

- starts processes
- ends processes
- controls resource usage
- schedules cpu time
- allows inter-process communication

Context switching

1. P1 is executing
2. capture PCB(1)
3. load PCB(2)
4. P2 is executing

Parallelism and Concurrency

- **concurrency**: multiple processes are being worked on, not necessarily simultaneously
- **parallelism**: multiple processes execute simultaneously on different CPU cores

Threads

- independent sequences of execution on the same process
- Context switching between threads is *efficient*
- share memory space
- have own execution stack
- have own instruction stream

THREAD ATTRIBUTES

- id
- name
- priority
- status (new, runnable, blocked, waiting, terminated, etc.)

Threads in Java

- the console is shared between all threads, internal mechanisms make sure only one thread read/writes at the same time. who gets access is **random**
- Threads can **continue to run even if main() returns** already

OPTION 1

Override run method and call `.start` to create the thread.

```

class ConcurrWriter extends Thread {
    public void run() {
        // if multiple threads started -> concurrent execution
    }
}
ConcurrWriter writerThread = new ConcurrWriter();
writerThread.start(); // calls ConcurrWriter.run()

```

OPTION 2

```

public class ConcurrWriter implements Runnable {
    public void run() { ...
        // if multiple threads started -> concurrent execution
    }
}

ConcurrWriter writerTask = new ConcurrWriter(); // task
Thread t = new Thread(writerTask); // executor
t.start(); // calls ConcurrWriter.run()

```

03 Thread Join

Busy waiting: Main thread überprüft laufend ob das Resultat der worker-Threads schon da sind. Braucht CPU Zeit.

Java Join

- instead of constantly checking, let main thread go to sleep and wake up when results ready
- join produces overhead, so if worker threads are quick, busy waiting might be better
- f.ex. in the main thread, call `worker.join()` to make main sleep until worker finishes
- `thread.join()` may throw an Exception, so put in try catch

Exceptions

- In a sequential program an exception terminates the program, if not caught.
- if a worker thread throws exception
 - exception is shown on console
 - `thread.join()` is unaffected
 - main thread may not be aware

Interrupt Threads

Threads können sich immer nur selbst interrupten. Aber möglich ist ein Interrupt flag, dass der andere Thread lesen und reagieren kann.

```
public class InterruptDemo {  
  
    static class MyTask implements Runnable {  
        public void run() {  
            while (!Thread.currentThread().isInterrupted()) {  
                // keep running  
            }  
            System.out.println("Stopped!");  
        }  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
  
        Thread t = new Thread(new MyTask());  
        t.start();  
  
        Thread.sleep(100);  
        t.interrupt();    // set interrupt flag  
    }  
}
```

04 Shared Resources

→ 05 Slides

| Terminology and Introduction

Begriff	Beschreibung
Data Race	Ein Thread schreibt, während ein anderer Thread gleichzeitig denselben Ort liest oder beschreibt.
Deadlock	Mehrere Threads warten darauf, dass der jeweils andere eine Ressource freigibt. Zyklische Abhängigkeit (∞ wenn aufgemalt). Directed Graph describing relation or Threads and Locks has a cycle.
Livelock	In Livelocks the system makes no progress, although the threads execute statements/use CPU time. In deadlocks, no statements are executed.
Bad Interleaving	Die zeitliche Abfolge der Schritte verschiedener Threads führt zu falschem Endergebnis, selbst wenn die einzelnen Schritte an sich korrekt sind.
Critical Section	Ein sensibler „Einer-nach-dem-Anderen“-Bereich im Code, in dem auf gemeinsam genutzte Daten zugegriffen wird.
Mutual Exclusion	Ein 2. Thread kann keinen kritischen Abschnitt betreten, bevor der aktuelle Thread ihn verlassen hat. Synchronize blocks create mutual exclusion.
Atomicity	Eine Gruppe von Operationen, die unteilbar ausgeführt werden. Kein anderer Thread kann die Daten in einem halbfertigen Zustand sehen.

synchronized

Warning

NIE `synchronized` auf einen Boxed type wie z.B. `Integer`. Problem ist, bei `x+=1` erstellt Java ein neues Objekt

Wenn mehrere Objekte von/auf die gleiche Quelle readen/writen, dann ist ja die Reihenfolge random. Mit `synchronized` können wir aber Code-Blöcke festlegen, die definitiv in der Reihenfolge (und nacheinander) ausgeführt werden.

Während `synchronized` ausgeführt wird, bekommt einzig **der aktuelle Thread** den Zugriff auf die gemeinsame Quelle. Die Quelle ist **locked**. Nachdem `synchronized` fertig ist, wird der lock freigegeben und andere Objekte können diesen "aquiren".

When we perform a write on a thread with shared memory, do so under a lock.

Wenn wir `public synchronized run() {}` zur Methode schreiben synchronisieren wir auf `this`. Wenn wir auf ein anderes Objekt synchronisieren wollen, schreiben wir

```
public void run(){
    synchronized (x) {
        // code
    }
}
```

Code example

Recursive Locks, Interleavings

- If a thread tries to acquire a lock that is already taken, it becomes blocked and must wait until woken up. See the [Life cycle of a thread](#) for reference.
- Java locks are reentrant. A thread can request a lock even if it has already and it won't throw an error. It will increase the **lock count**, and we will have to release the lock **twice too**.

Recursive Locks

The lock belongs to the thread, not to the method. This means that **the thread carries the lock with it** as it moves through method calls, rather than the lock being tied to a specific method.

So methods can pass around the lock without releasing it.

```
public class Foo {  
    public synchronized void f() { ... }  
    public synchronized void g() { ... f(); ... }  
}
```

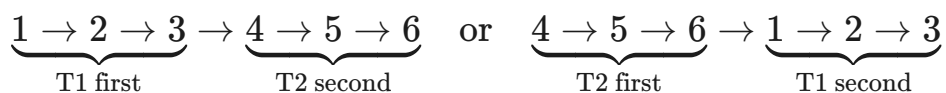
Interleavings

Without `synchronized`, all instructions are shuffled ("interleaved") in each other. With `synchronized` we can control the possible interleavings.

Both threads must **lock on the same object to protect the same shared resource**

EXAMPLE

Both T1 and T2 lock on `this`, so they **cannot run at the same time**. One must finish completely before the other starts. This limits the possible interleavings to only two:



Wait, Notify, NotifyAll

Example: each runner can start after the previous runner finished (except the first one)

Warning

NIE synchronized auf einen Boxed type wie z.B. Integer . Problem ist, bei `x+=1` erstellt Java ein neues Objekt!

Producer-Consumer

Often one part of the system generates work, another part processes it, buffer stores it for passing it on (f.ex. through a linked list):



Annahme mehrere Consumers machen:

```
while (buffer.isEmpty()); // spin-wait
    buffer.remove();
```

Dann könnte ein Consumer zwischen dem `.isEmpty()` check und dem `remove()` das letzte Element `remove()`, dann ist `.remove()` ungültig da leer. Selbst wenn nur eine Anweisung; z.B. `.remove()` sind **mehrere Anweisungen im Bytecode**.

Synchronize? → **Deadlock**

Stattdessen (richtig):

- `while(true)` means they constantly try to get access to the buffer
- `synchronize` means the actions in the block are executed in their order. After that block the lock is released and others can acquire it again, f.ex. through `while(true)`
- `buffer.wait()` give up lock mid `synchronize`. Makes Thread sleep until `notification` comes in, then picks up from where it went to `wait()`. But there's no guarantee that Consumer will get the lock after `.notifyAll()`.

Producer adds an item and calls `notifyAll()`. Wakes up all threads currently sleeping in `wait()`. They all become `RUNNABLE` again and compete for the lock. Whoever wins re-checks `isEmpty()` via the `while` loop and either proceeds or goes back to sleep.

`notifyAll()` instead of `notify()`: multiple consumers, `notify()` might wake wrong thread, `notifyAll()` wakes everyone and lets them sort it out. `notify()` wakes up a random thread.

Warning

`while(condition) { counter.wait() }`, NICHT `if`. This is because sometimes **Threads get woken up randomly**.

WAIT VS JOIN

	<code>wait()</code>	<code>join()</code>
Called on	any object (the lock)	a <code>Thread</code> object
Purpose	wait for a condition to become true	wait for a thread to finish
Woken up by	<code>notify()</code> / <code>notifyAll()</code>	the thread terminating
Releases lock?	Yes	No
Used inside <code>synchronized</code> ?	Yes, required	No

05 Hardware

Not important

This chapter can be skipped/speedrun, I recommend jumping to [06 Pipelining](#) instead.

- Modern CPUs exploit parallelism internally (caching, vectorization, ILP, pipelining), even on a single core
- Code structure determines how effective these structures are

History and Preamble

- Moore's Law: Transistor counts on a chip double approximately every two years
- Until the early 2000s: Shrinking transistors enabled both higher transistor densities and higher clock speeds, directly driving computational power growth
- computer executes one instruction per clock. An instruction describes an operation for a processor to perform (modify the computer's state)

Single Core

Instruction level parallelism

These optimizations make control unit more complex: Out of order scheduling, branch prediction, memory prefetching, cache, etc.

But limits: - Power consumption and heat - ILP wall - Memory wall

So we got

Multi Core

Thread level parallelism

We need to parallelize tasks, so **Programmers** must **design software to split work** into independent threads.

Caching

Von-Neumann-Architektur

Memory access results in latency → Invention of **Cache**

Example, summing an Array

Best performance in **sequential access**, efficient due to cache locality, cache prefetching works

Jumping around the Array is **Cache-unfriendly**, causing cache misses because prefetching is inefficient

→ a computer has multiple caches. Here each L stands for a cache, f.ex. L1 is 5x faster than L2, etc.

Compute optimization

3 Approaches to apply parallelism:

- Vectorization
- ILP
- o6 Pipelining

Vectorization

Verschiedene, unabhängige Operationen vom gleichen Typen können parallelisiert werden, durch Optimierungen vom Compiler

SIMD

SIMD: Single Instruction, Multiple Data. One instruction broadcasts to multiple ALUs (calculation units)

Compiler sieht Optimizationsmöglichkeiten

ILP

Processor finds independent instructions and executes them in parallel ("Superscalar"). Independent: result of one instruction isn't input of the other. Hardware enables ILP

ILP: Happens on a single thread, so no concurrency.

Multi-threading: multiple cores

Combination: Thread scheduled on a core, ILP to separate on the same thread

Prefetching: Preloading instructions into the CPU cache before needed.

```
x = a + b; // this and the one below are independent
y = c + d; // can execute these 2 instructions in parallel
z = x * y; // this one depends on results above, so has to wait
```

IPL can also ****reorder** statements**, if independent from one another.

Independent Statements

Two statements are **independent**, if - different register names - different memory addresses

Slides - [Slide 1](#) - [Slide 2](#) - [Reordering](#)

06 Pipelining

→ split a "process" into different stages, each with clear function, f.ex. clothing washing process

Balanced

Balanced pipeline: all steps require same time

- 1. Instruction Fetch** - CPU reads the instruction from memory - may prefetch multiple instructions
- 2. Instruction Decode** - decode the instructions, prepare instructions for execution - instructions are then dispatched to the correct execution units
- 3. Execution** - CPU executes decoded instruction - Superscalar execution: instructions dispatched to multiple execution units. - SIMD: apply one **s**ingle **i**nstruction to **m**ultiple pieces of **d**ata all at once
- 4. Memory Access** - If needed, exchange data between CPU and memory - pre-fetching, caching, etc.
- 5. Writeback** - Save the final result of the execution - outcome of the execution stage is written back into the CPU's registers - Superscalar Execution: Multiple instructions can write their results back to different registers or memory locations at the same time

Throughput

Throughput = number of instructions that exit the pipeline per time unit. higher is better

→ EINHEITEN angeben

- For a specific number of instances n : $\text{Throughput} = \frac{n}{\text{total time}}$ oder $\frac{\text{total time}}{n}$
- For infinite pipeline (per execution unit)

$$\text{Throughput bound} = \frac{1}{\text{Dauer vom längsten Schritt}}$$

Beispiele: - $t_{\text{put}} = \frac{1}{3s}$ means 1 instruction per 3s - 1 Produkt pro 15 min

Latency

Latency = time needed to perform single computation. **lower is better**

- Latency ist nicht konstant, kann sich verändern
- **Latency bound** = (no of stages) · Dauer vom längsten Schritt
- Konstant: nehme einfach erste Instanz `latency = Zeit für erste Instanz`
- Nicht konstant, für die **n-te Instanz**: `latency = Zeit für erste Instanz + (Längste Stage - Time of first stage) * (n-1)`

Total time

Example, we want to know how much time is needed for 100 instances.

Suppose in this example we are processing images. Every image has a sequence of operations.

1. Calculate for first instance. $T_{\text{first}} = 40 + 50 + 30 + 40 = 160 \text{ ms}$
2. Once the pipeline is full, it outputs one finished image at the rate of its bottleneck.
 $T_{\text{remaining}} = 99 \times 50 \text{ ms} = 4950 \text{ ms}$
3. $T_{\text{total}} = 160 \text{ ms} + 4950 \text{ ms} = 5110 \text{ ms}$

Example: unbalanced Pipeline creates waiting delays

Example: washing loads

→ make pipeline balanced by increasing time for each stage to match longest stage

→ add 2 dryers working in a row

07 Scalability

How does performance improve with additional cores? More cores/hardware do not always result in a linear speedup.

Limiting factors

- parts of a program that cannot be executed parallelly
- data structures that require sequential access (f.ex. linked list)
- Verteilung auf Cores und Threads
- Overhead and time for synchronization/locking
- Memory bandwidth, cache misses

Speedup

S: Speedup, How performance improves with additional cores

T: Execution time

Success

T_1 : sequential execution time T_p : Execution time on p CPUs.

$T_p = \frac{T_1}{p}$: linear speedup

$T_p > \frac{T_1}{p}$: normal speedup

Parallel speedup on p CPUs

$$S_p = \frac{T_1}{T_p}$$

- $S_p = p$: linear - $S_p < p$ (sub-linear, performance loss)

Efficiency

$$E = \frac{S_p}{p}$$

Example

Beispiel

80% parallelisierbar, 20% sequenziell.

- $T_1 = 10$
- $T_{seq} = 10 \cdot 0.2$
- $T_{par} = 10 \cdot 0.8$
- $p = 8$ CPUs

$T_8 = ?$

$$T_8 = T_{seq} + T_{par} \quad (1)$$

$$= 10 \cdot 0.2 + \frac{10 \cdot 0.8}{8} \quad (2)$$

$$= 2 + 1 \quad (3)$$

$$= 3 \quad (4)$$

$S_8 = ?$

$$S_8 = \frac{T_1}{T_8} = \frac{10}{3} = 3.33$$

$E = ?$

$$E = \frac{3.33}{8} \approx 0.4 \implies 40\%$$

Amdahl's Law

"Will ein Problem lösen, geht nimmer schneller"

The speedup from parallelization is limited by the part that must run sequentially.

W_{ser} ... time spend on non-parallelizable work W_{par} ... time spend on parallelizable work

$$T_1 = W_{ser} + W_{par}$$

$$T_p \geq W_{ser} + \frac{W_{par}}{p}$$

So in the definition of speedup:

$$S_p = \frac{T_1}{T_p} \leq \frac{W_{ser} + W_{par}}{W_{ser} + \frac{W_{par}}{p}}$$

Formula with percentage

f is the percentage of how much is sequential

$$S_p = \frac{1}{f + \frac{1-f}{P}}$$

Gustafson's Law

"ich kann mehr (anderes) in der selben Zeit machen" Parallel part of a program scales with the problem size. Wie viel Arbeit können wir in einem fixen Zeitfenster ausführen, wenn wir mehr Hardware hinzufügen?

Formulas

- f : Percentage of sequential part
- $1 - f$: parallelizable part
- P : number of cores
- T_{wall} : total time allowed for execution
- W : amount of work
- S_p : speedup achieved by p processors

$$W = P \cdot (1 - f) \cdot T_{wall} + f \cdot T_{wall}$$

Success

$$S_p = f + P \cdot (1 - f) \quad (5)$$

$$= P - f \cdot (P - 1) \quad (6)$$

Siehe #exam , Kosten-Nutzen Analyse, lohnt es sich, mehr Hardware hinzuzufügen?

08 Arbeitsaufteilung

Was, wenn ein Thread sehr viel länger braucht als die anderen? Dann funktioniert `fork/join` nicht mehr wirklich effizient.

Divide and Conquer

```
- if cannot divide:
  - return unitary solution (stop recursion)
- divide problem in two:
  - solve first (recursively)
  - solve second (recursively)
- combine solutions
- return result
```

Code example: Summing an Array

- Wenn ein Thread nicht erstellt werden kann aufgrund mangelnder Ressourcen:
Exception, out of Memory
- Achtung auf richtige `start/join` Reihenfolge, sonst ist es vielleicht sequentiell
- due to **thread overhead**, sequentially creating a thread for every operation is inefficient.
 - use a sequential cutoff, f.ex. when summing, around 500-1000
 - do not create two recursive threads; create one and do the other work on the thread that initiates the other thread

Zuteilung von Task auf Threads

Zur Arbeitsaufteilung, also der Zuteilung von Tasks zu Threads nutzen wir **Java Frameworks**, z.B.

- den Executor Service, oder - das Fork/Join Framework

Framework: Executor Service

Warning

Problem vom Executor Service: Threads blockieren. Daher **nicht verwenden für `**rekursive**` Probleme**. Für flat structures or tasks that can run independently in parallel gut.

1. Main thread submits task to the Executor Service
2. Main thread immediately gets back a `future` by the Executor Service. The future will later hold the result once the calculation is done.
3. Executor Service calculates in parallel
4. the Main thread can at any time access the future and see if the result of the calculation is stored there already

Callable vs Runnable

```
.submit(Callable<T> task) → Future<T>  
.submit(Runnable task) → Future<?>
```

→ Callable hat einen return

Executor Service Syntax (Hello World)

```
int ntasks = 1000;  
ExecutorService exs = Executors.newFixedThreadPool(4);  
  
for (int i = 0; i < ntasks; i++) {  
    HelloTask t = new HelloTask("Hello from task " + i);  
    exs.submit(t);  
}  
  
exs.shutdown(); // initiate shutdown, does not wait, but can't submit more tasks
```

```
static class HelloTask implements Runnable {  
  
    String msg;  
  
    public HelloTask(String msg) {  
        this.msg = msg;  
    }  
  
    public void run() {  
        long id = Thread.currentThread().getId();  
        System.out.println(msg + " from thread: " + id);  
    }  
}
```

Framework: Fork Join

- recursiveAction has return type void
- recursiveTask has a return value
- fork/join only from within the pool, invoke externally

→ when a task is waiting, it is suspended and **other tasks are allowed to run** on that thread. Tasks on the thread are exchanged such that it always runs.

Wie beim Framework Executor Service lesen Threads von der `global queue`. Neu: jeder Thread hat eine `local queue`.

Jeder Thread arbeitet die `local queue` in LIFO (last in first out) ab. Also stellt man sich die Queue als Röhre vor, kommen neue Aufgaben (bei einer Rekursion also zuerst die grösseren Aufgaben) von links hinein, und werden per LIFO auch von links abgearbeitet.

Ein Thread kann auch auf die `local queue` von einem anderen Thread zugreifen, in FIFO (also von hinten der local queue des anderen Threads). Eine hintere Aufgabe ist also eine aufwendigere (gut für Lastverteilung).

Extends `RecursiveTask<Integer>` und die `compute()` methode statt run.

```
ForkJoinRecursiveSum left = new ForkJoinRecursiveSum(arr, lo, mid);
ForkJoinRecursiveSum right = new ForkJoinRecursiveSum(arr, mid, hi);

left.fork();
right.fork();

int leftAns = left.join();
int rightAns = right.join();

// Better:
//left.fork();
//int rightAns = right.compute();
//int leftAns = left.join();
```

09 Task graphs

Graph ohne cycles

Introduction

- a program using fork/join can be seen as a DAG
- Vertices: units of work
- Edges: dependencies (source must finish before destination starts)

fork - current node ends, 2 outgoing edges and vertices created. If possible, then in parallel 1.

The newly forked task 2. The continuation of original task

join - one node with 2 incoming edges - 1 edge from the forked task that finished - 1 edge from continuation of original task that was waiting for the forked task - program cannot proceed until both incoming branches complete

Example, Fibonacci

Performance Model

Introduction

T_p : execution time on P worker threads. This does not necessarily mean physical hardware cores.

To determine how much a program can potentially speed up by running in parallel, the model relies on two critical metrics derived from the DAG.

Speedup:

$$S_p = \frac{T_1}{T_p}$$

Maximum Parallelism

$$S_\infty = \frac{T_1}{T_\infty}$$

Work: T_1 - execution time if sequentially run - on DAG, total number of nodes in the entire graph.

Span, Critical Path: T_∞ - longest chain of dependent operations in the DAG (tasks that cannot be run simultaneously)

Width of the Graph: number of processors to archive T_∞ , so the maximum speed

DAG Bounds

wide graph: higher potential parallelism (shorter T_∞) **deep graph:** more dependencies, less speedup

LOWER BOUND

Work law: linear speedup

$$T_p \geq \frac{T_1}{p}$$

Span law the longest sequential chain determines the time, no matter how many worker threads we have

$$T_p \geq T_\infty$$

Combining Work and Span law:

$$T_p \geq \max\left(\frac{T_1}{p}, T_\infty\right)$$

UPPER BOUND

The total time should not exceed the time it takes to do the perfectly divided work (T_1/p) plus the time spent waiting on the critical path's dependencies ($\mathcal{O}(T_\infty)$).

$$T_p \leq \frac{T_1}{p} + \mathcal{O}(T_\infty)$$

Execution times with ForkJoin

Asymptotic Bound

Summing an Array: - Sequential: $O(n) = T_1$ - Parallel: $O(\log n) = T_\infty = \text{span}$ - Parallelism: $O\left(\frac{n}{\log n}\right)$

Patterns

Reduction always achieve $O(\log n)$ Map achieves $O(\log n)$ with Divide & Conquer

Reduction

The dimension (size) of the output is smaller than of the input. We get the output by applying an **associative operator** on all Input entries.

Maps

A map applies a function to each element. f.ex. squaring each element of an array

$|output| = |input|$

Zip map: multiple inputs, f.ex. element-wise addition of 2 arrays **Stencil:** multiple inputs of the same array to calculate the output. f.ex. `0[i] = f(Input[i-1], Input[i])`.

Prefix-sum

```
fromLeft = sum_of_left + fromLeft of top
```

$O(n)$ work, $O(\log n)$ span

Pack

Given an array input, produce an array output containing only elements such that $f(\text{element})$ is true.

Output array hat Elemente aus dem Input mit bestimmten Eigenschaften. Erkennen: Maps. In neues Array schreiben: Prefix-sum.

Data structures

For parallelism: **balanced trees** / **arrays** are better than lists, as we can bet all data in $O(\log n)$ instead of $O(n)$.

Example, Quicksort

10 External Locks, Race Conditions

External Locks

- nutze `private Lock lk = new ReentrantLock()` in der BankAccount Klasse. Dann am Anfang der Methode `lk.lock()`, im finally `lk.unlock()`. So wird wirklich auf diesen BankAccount ge-synchronized und nicht auf die methode (unlike `synchronized`)
 - try finally mit unblock im finally damit wenn crash dann trotzdem unlocked
 - oft ist Reihenfolge vom locken wichtig um deadlock zu vermeiden (suppose eine Überweisung locked A dann B, andere von B nach A). Manchmal muss auch alle Threads locken.
-

Race Conditions

Race Condition: Berechnungsergebnis hängt von Ausführungsreihenfolge ab.

Low-level race conditions: data races - fehlerhaftes Programm, da mehrere Threads auf geteilte Ressource zugreifen (lesen/schreiben) → error - [10 Slides.pdf](#)

High-level race conditions: bad interleavings - fehlerhaftes Programm, da falsche Ausführungsreihenfolge trotz guter Synchronisation von Ressourcen - [10 Slides.pdf](#)

Peek

Looks at the top element of a stack without changing anything (`pop` , then `push`). `Push` and `pop` are synchronized in java, but our peek function per default is not → bad interleavings possible.

Solution: `synchronize` entire `peek` function.

```
class Stack<E> {  
    ...  
    synchronized E peek() {  
        E ans = pop();  
        push(ans);  
        return ans;  
    }  
}
```

11 Guidelines

Eine Speicherlocation ist mindestens eines von

- Thread-local
- Immutable
- Synchronized

Guidelines

1. **Keine Data Races** (nicht mehrere Threads können den gleichen Speicherort lesen/schreiben)
2. Nutze **locks** wenn synchronisation needed, either
 1. Coarse grained: more objects per lock, fewer locks
 2. Fine grained: more locks, fewer objects per lock
3. **keine teuren Berechnungen in critical sections**
4. think, what **operations need to be atomic** (and create locks that way)

Part 2

Locks · Synchronization · Lock-Free · Consensus · TM

01 Hardware Revamped

Introduction

- 2 threads with k statements each: $\binom{2k}{k}$ possible interleavings (ziehen ohne zurücklegen)
- showing that something isn't possible by showing what needs to happen before something and through transitivity find a contradiction (a circle when drawn)
- parallel threads interacting via shared memory depends on **hardware, runtime system, and programming language**.
- Compiler, CPU and memory reorder for better performance (works sequentially, might break in parallel)
- possible solutions for bugs caused by cpu based memory optimizations
 - synchronized (slow)
 - volatile (slow)
- Architekturen heutzutage (x86, AMD64, ARM, etc.) sind **nicht sequentiell konsistent**. Umordnung kann also vorkommen. Als Lösung:

Java Memory Model (JMM)

Welche Garantien über die Ausführungsreihenfolge habe ich?

Abbreviation	Full Term
JMM	Java Memory Model
PO	Program Order
SA	Synchronization Action(s)
SO	Synchronization Order
SW	Synchronizes-with
HB	Happens-Before

Your code runs in Program Order when seen a single thread in isolation. Because the CPU reorders things, Synchronization Actions create a Synchronization Order. When threads interact with these actions, they create a Synchronizes-with handshake. This handshake establishes a

Happens-Before relationship, which is the only way to guarantee that Thread B actually sees what Thread A just did.

Warning

Rescheduling on a single Thread can only happen in a way that the programmer doesn't notice.

Program Order

The order of statement execution on a single thread. The JMM can re-order though, this is about the order of execution. **Total order on one thread, but not across threads** (partial order on threads).

Synchronizes-with

Two threads "touch" the same point on that master timeline through a Synchronization Action (like reading and writing the same `volatile` variable), creating a **handshake**.

A write to a `volatile` variable `v` *synchronizes-with* every subsequent read of `volatile` variable `v` by any thread.

SYNCHRONIZATION ACTIONS

Critical actions that need a fixed order.

- Read/write of **volatile variable**
- Lock/Unlock monitor (what is a monitor?)
- First/last action of a thread (synthetic)
- First/last action, starting, or terminating a thread

Synchronization Order

Global order of all Synchronization Actions.

- **Total Order:** every thread in the application agrees on this order
- **Consistency:** If you look at the SO, read/write of a `volatile` var will make sure all variables that were updated before this statement are up to date.

Happens-Before

- transitive closure of program order and synchronizes-with order (rote Linie durch den ganzen code)
- everything Thread A did before the handshake is guaranteed to be visible to Thread B.

HB Consistent

Read - The **most recent write** in its happens-before chain, **OR** - Any **unordered write** (a write with no HB relationship to the read).

HB Inconsistent

Read - An **outdated write** (one already overwritten by a newer write in the HB chain), **OR** - A **future write** (one that happens-before the read)

02 State Space Diagram

Critical sections (locks)

1. Mutual exclusion (statements not interleaved)
2. Freedom from deadlock: a process trying to enter a critical section will eventually succeed
3. Freedom from starvation: all processes trying to enter a critical section will eventually succeed

Mutual Exclusion Problem

State space diagram

F.ex. state p4 means the next step is p4.

- **Mutual exclusion**: gibt keinen state mit p3 und q3
- **Keinen Deadlock**: gibt keinen state ohne ausgehendem Pfeil
- **Keinen Livelock**: gibt keinen unendlichen Zyklus, der die Critical Section auslsst
- **Kein Dead/Livelock**: gibt von jeder Node Pfad zu critical section
- **Freedom from individual starvation**: jeder thread der die critical section haben mchte, bekommt sie irgendwann

03 Software Locks

- Um einen Lock fr n Prozesse zu bauen, bentigt man im Speicher mindestens n Variablen, Speicherbedarf $O(n)$ - Achtung volatile Array doesn't work (only pointer gets atomic)

Success

Fair + Deadlock-Free = starvation-free

Decker's Lock

Peterson Lock

- We have a flag that indicates we are interested in entering the CS
- I'm the victim, indicate that other thread goes first
- Wait until other thread no longer interested
- CS
- indicate no longer interested

Eigenschaften - correct - fair

Proofs

- **Mutual exclusion:** contradiction. Assume both CS_P and CS_Q (critical sections). By chaining the precedence of write (W) and read (R) events, it shows a logical contradiction where transitivity breaks down, proving that concurrent access is impossible
- **Freedom from Starvation:** contradiction. Assume Process P is stuck forever in its `while` loop. It then evaluates every possible state Process Q could be in (ignoring the lock, constantly acquiring and releasing it, or also stuck). Every single path leads to a mathematical contradiction, proving P cannot be starved indefinitely.

Filter Lock

Extension of Peterson's lock to n processes

- mutual exclusion
- deadlock free
- starvation free
- not fair (first-come-first-serve)

The filter filters out threads such that only one ends up in the CS. Every thread knows its level in the filter. Each level uses Peterson to filter, there's a victim per level that lets other pass when conflict. That happens on each level, so only one ends up.

Fairness

Filter lock is NOT fair (first come first serve in CS).

Bakery Lock

- Wartezimmer mit gezogenen tickets
 - Fair
1. Thread will in CS
 2. Schaut auf Nummern von anderen Threads, gibt sich Nummer +1 von max der existierenden threads
 3. Thread-ID als backup falls 2 Threads gleiche Nummer

04 Hardware Locks

Atomic hardware operations - **Test and Set** (set 1 if currently 0) - **Compare-and-Swap** (wenn wert im speicher gleich argument alt, setze wert im speicher auf neuen wert) - Load Linked / Store-Conditional (ARM).

Time per thread rises with number of threads

| Locks with TAS

- both threads try TAS in while loop, only one sets it to 1 the other gets 0, needs only one memory location
- problem, all threads spam TAS in while loop
- Backoff
 - threads fight for access to same resource slows down
 - solution: **go to sleep with random duration** → less try access at the same time
 - double waiting duration each time the resource is not free, reset when accessed

TATAS

- TAS with Backoff still slow
- TATAS also
- **Solution:** add flag, read flag if lock is free (fast)
- only if free, start TAS, else go to sleep (backoff)

05 Deadlocks

Deadlock detection: Directed Graph describing relation of Threads and Locks has a cycle. Suppose A waits for B, B waits for C, C waits for A.

- `java.util.concurrent.atomic.AtomicBoolean`
- `boolean set();`
- `boolean get();`
- `boolean compareAndSet(boolean expect, boolean update)`
- `boolean getAndSet(boolean newValue)`

Bank Account, Transfer from A to B - non-overlapping smaller critical sections means money is gone for a short time, so no atomarity - one global lock, bad performance - Solution: global ordering:

06 Semaphores

Semaphoren erlauben mehreren Threads Zugriff bis zu einer festen Grenze. Sie funktionieren wie ein Zähler für verfügbare Plätze.

Locks haben entweder 0 oder 1 Thread in der CS. Semaphoren haben bis zu `s` Threads in der CS.

Vgl. Parkhaus mit Kapazität `s`

- `acquire(S)` to `dec(S)` counter
- `release(S)` to `inc(S)` counter

Beispiel

1. Thread A führt aus
2. Thread A macht `release(S)`, also erhöht sich der counter
3. Thread B führt aus
4. Thread B stösst auf `acquire(S)`. Wenn Thread A noch nicht `release` ausgeführt hat ist der counter 0, somit wird B schlafengelegt. So wartet B mit der letzten Operation bis A fertig ist. Wenn A schneller war ist der counter bereits erhöht, somit kann B den counter decrementen und fortfahren.

```
public class MySemaphore {
    private int count;
    public MySemaphore(int maxCount) {
        count = maxCount;
    }

    public void acquire() throws InterruptedException {
        synchronized (this) {
            while (count == 0) this.wait();
            count--;
        }
    }

    public void release() {
        synchronized (this) {
            count++;
            this.notify();
        }
    }
}
```

07 Barrier

Barrieren lassen Threads warten, bis alle an einem gemeinsamen Punkt angekommen sind. Erst dann dürfen alle weiterlaufen. Rendezvous der Semaphoren ist wie eine Barriere, erst wenn beide hier sind, können wir post ausführen

Alle Threads kommen zum rendezvous Punkt der Semaphore, und erst dann laufen sie weiter.

Turnstile

- acquire, dann gleich release
 - schritt für schritt releasen sich die Threads gegenseitig
-

Beware in implementation - count++ is not atomic, need to protect with acquire/release mutex - barrier needs to be reusable, meaning count 0 at end - only one thread should trigger release/acquire, not all at once

With Monitor:

reusable with a double-door setup

```
synchronized void await() throws InterruptedException {
    while (draining) {
        wait();
    }
    ++i;
    while (i < n && !draining) {
        wait();
    }
    if (i-- == n) {
        draining = true;
        notifyAll();
    }
    if (i == 0) {
        draining = false;
        notifyAll();
    }
}
```

reusable, worse

```

public class Barrier {
    final int threads;

    private int count = 0;
    private int generation = 0;

    public Barrier(int threads) {
        this.threads = threads;
    }

    public synchronized void await() throws InterruptedException {
        int myGeneration = generation;
        count++;

        if (count == threads) {
            count = 0;
            generation++;
            notifyAll();
        } else {
            while (myGeneration == generation) {
                wait();
            }
        }
    }
}

```

not reusable

```

public class MyBarrier {

    private final int limit;
    private volatile int count;

    MyBarrier(int n){
        this.limit = n;
        this.count = 0;
    }

    synchronized void await() throws InterruptedException {
        count++;

        while (count < limit) {
            wait();
        }
        notifyAll();
    }
}

```

With Semaphore:

```

import java.util.concurrent.Semaphore;

public class MyBarrier {
    private final int n;
    private int count = 0;

    private final Semaphore mutex = new Semaphore(1);
    private final Semaphore barrier1 = new Semaphore(0);
    private final Semaphore barrier2 = new Semaphore(1);

    public MyBarrier(int n) {
        this.n = n;
    }

    public void await() throws InterruptedException {
        // Phase 1: wait until all threads arrive
        mutex.acquire();
        count++;

        if (count == n) {
            barrier2.acquire();    // close second turnstile
            barrier1.release();    // open first turnstile
        }

        mutex.release();

        barrier1.acquire();
        barrier1.release();

        // Phase 2: wait until all threads leave phase 1
        mutex.acquire();
        count--;

        if (count == 0) {
            barrier1.acquire();    // close first turnstile
            barrier2.release();    // open second turnstile
        }

        mutex.release();

        barrier2.acquire();
        barrier2.release();
    }
}

```

08 Producer Consumer

- To rechnet x aus und gibt es an T1 weiter
- x muss nicht synchronisiert werden da immer nur ein Thread zugreift, müssen nur Zugriff managen

- Queue mit pointern für `out` und `in` position und **wrap-around Semantik** (das lineare array springt nach dem Ende an den Anfang ($\text{position} \bmod \text{länge}$))

Voll und leer erkennen: - Queue leer: `in` und `out` pointer gleich - Queue voll: wenn ich noch ein Element einfüge, ist `in` gleich `out`. Wir definieren also die Queue als "voll", wenn noch genau ein Platz frei ist

PC with Semaphores

2 Semaphoren zählen, ob der Puffer **nicht leer** oder **nicht voll** ist, sodass Consumer nur bei vorhandenen Elementen und Producer nur bei freiem Platz weiterlaufen.

Ein zusätzlicher binärer Semaphore schützt den gemeinsamen Puffer, damit immer nur ein Thread gleichzeitig `buf`, `in` und `out` verändert.

PC with Lock

ein gemeinsamer Lock schützt den Puffer, damit nur ein Thread gleichzeitig `buf`, `in` und `out` verändert

2 Conditions: Producer warten auf `notFull`, wenn der Puffer voll ist, und Consumer warten auf `notEmpty`, wenn der Puffer leer ist.

F.ex. in Producer / Consumer we have `notFull` and `notEmpty` conditions. Wir verwenden nicht mehr intrinsic lock, also kein `synchronized` sondern muss manuell `lock.lock()` mit `try { } finally { unlock }`

PC with Sleeping Barber

- Will nicht `notFull` und `notEmpty` nicht senden obwohl keine threads warten
- Idee Barber checks waiting room, sleeps. Client either enqueues or wakes barber. Achtung, wenn barber und client gleichzeitig schauen deadlock

Lösung

- add counters for clients and barbers.
- $m \leq 0$: buffer full, -m clients waiting
- $n \leq 0$, buffer empty, -n barbers waiting

09 Monitors

Wir unterscheiden jetzt das Konzept der *intrinsic built-in locks* eines Objekts und der external locks. Ein Monitor ist eine Verallgemeinerung des Konzepts der *intrinsic (synchronized) locks* in Java. Es ist eine Datenstruktur mit `wait / notify / notifyAll`.

Wie schon früher bekannt, gilt immer noch, es gibt - ein Entry set of threads: tries to acquire the lock. One thread succeeds. The thread can go to the - wait set (sleep state): by releasing the lock, then it can be woken up with notify by an active thread. Once woken up, it competes with the threads from entry set to get access of the lock, to become the - owner of the lock. - Release and exit removes the thread from the pool. If all threads are in waiting state, we have a deadlock.

Wir können das wait set unterteilen, um nur z.B. eine bestimmte Kategorie von Threads zu wecken. Wir verwenden das Condition Interface. F.ex. in a o8 Producer Consumer setup we create 2 wait sets.

Arten, Locks weiterzureichen (irrelevant)

Es gibt unterschiedliche Varianten, wie ein Lock weitergereicht werden kann (irrelevant)

signal and continue (java) 1. signaling process continues running 2. signaling process moves signaled process from wait set to entry set

signal and wait 1. signaling process gives lock to signaled process 2. signaling process exits monitor and **goes waiting entry queue**

signal and exit 1. signaling process gives lock to signaled process 2. signaling process exits monitor entirely

Condition interface of Monitors

Wenn wir manuell ein lock erstellt haben, können wir die conditions verwenden um Gruppierungen zu erstellen. Es ist immer noch nur **ein Lock**, aber Gruppierungen in der waiting queue.

Mehr flexibilität haben wir mit reentrant locks. `final Lock lock = new ReentrantLock();`

```
final Lock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
```

Jedes Java objekt hat `.wait()`, also hat man die wait, notify und notifyAll methoden umbenannt hier, um zu unterscheiden.

- `.await()`
- `.signal()`

- `.signalAll()`

PC with Lock (Monitors)

F.ex. in Producer / Consumer we have `notFull` and `notEmpty` conditions. Wir verwenden nicht mehr intrinsic lock, also kein `synchronized` sondern muss manuell `lock.lock()` mit `try { }` `finally { unlock }`

10 Reader Writer Locks

separate locked for writing from locked for reading

- $0 \leq \text{writers} \leq 1$
- $0 \leq \text{readers}$
- $\text{writers} * \text{readers} == 0$

This is needed f.ex. in Producer Consumer, if reader are much faster than writers, and we want the writers also go have a turn. This is **not** fair.

To make them fair, use Wait/Notify with synchronized methods `acquire_read`, `acquire_write`, `release_read()`, `release_write`, and an int for writers count and readers count. Also, have int for number of writers that are waiting, and number of readers that are waiting. Also, `writersWait` int that stores how many readers we're letting to pass after a writer finishes. We set this by counting how many readers try to write (stored in number of writers waiting int).

11 Lock Granularity

Beispiel linked list

Coarse grained locking

`synchronized` add, remove, contains methods

Fine grained locking

jedes listenelement hat ein eigenes lock

Starter (Hand over hand locking)

- hand over hand: threads locks können sich nicht überholen, also statt vorzulaufen bis zu vorgänger von delete und dann 2 nodes zu locken (problem ist dass race condition beim lock acquiren, aber davor speichern wir state). hier ist die lösung hand over hand, also wir laufen mit 2 locks die nodes ab und checken jedes mal ob das nächste element schon das

gesuchte element ist. so haben wir die locks die wir brauchen safe, und es kann kein anderer thread uns überholen.

- handover lock when traversing, always only lock 2 nodes at the time
- lock both read and write node, so for f.ex. `remove(c)` we need to lock b and c, as we move the pointer from b to d instead of b to c

Problems: - many lock/unlock because of traversal - locks can't overtake in the sense of making changes further to the right

first element make new dummy element in case that we want to insert right at the beginning

Optimistic synchronization locking

1. go to node without locking
2. validate all is okay (to make sure in the meantime, so after reading states and before having both locks, another thread hasn't changed something),
3. lock only needed nodes for method

validate: rescan: traverse from the start to check if the connection up to **the element after b** is still reachable (connected). Element after b is to check that 1) b ist still reachable and hasn't been deleted, and 2) b.next hasn't changed (f.ex. another thread hasn't deleted b.next, this is important for f.ex. adding)

```
private Boolean validate(Node pred, Node curr) {
    Node node = head;
    while (node.key <= pred.key) { // reachable?
        if (node == pred)
            return pred.next == curr; // connected?
        node = node.next;
    }
    return false;
}
```

Problems: - need to traverse list multiple times - not starvation free

Lazy synchronization locking

- like optimistic, but
- add deleted flag instead of actually directly deleting
- lazy delete nodes flagged as to be deleted when f.ex. traversing

delete c - lock b and c - check if b or c got marked and if the next pointer still stands (for add(c), this is our new validate instead of iterating from the start) - if not marked, mark c (flag) and then delete c (update pointer)

contains - iterate linked list while current key < destination key - return yes if both - current key = destination key - current key is not marked

Skip lists (ev. nicht prüfungsrelevant)

las vegas

- probability on height $n = 0.5^n$, we start at height 0
- left $-\infty$, right is ∞ , in between are values (list items)
- higher level lists are contained in lower lists, so level 0 contains everything
- logarithmic search
 - we traverse from left to right on the same level and go down when we are at a place where we are smaller than destination and next element is bigger than destination
 - we adapt the limits to narrow down on elements that are bigger or smaller than the destination
 - Example of finding 8:
- add
 - find predecessors lock free
 - lock predecessors
 - validate
- remove
 - find predecessors
 - change pointers to point to next value (next col in same row), OR mark as removed per lazy delete
 - lock predecessors, validate, physically remove, unlock

12 Lock Free

Concepts of Lock Free

- **Lock-free**: always deadlock-free, failure of one thread cannot cause failure of another thread. But we can have starvation
- **Wait-freedom**: freedom of starvation. wait freedom $\Rightarrow$ lock freedom
- readSomething() können wir einfach ohne locks machen
- writeSomething() mit CAS

Remember CAS (Compare and Swap): wenn wert im speicher gleich argument old, setze wert im speicher auf argument new. true means change was needed.

```
do {
    } while {
        // if we have been interleaved, try again
        !atomicReference.compareAndSet(oldObject, newObject)
    }
}
```

ABA Problem

Beispiel Stack

CAS suggests that no other thread has written between (a) and (c), but might be deceptive when changed and changed back again by another thread where the value now is the same but changes have been made. When an activity fails to recognize that a single memory location was modified temporarily by another activity and therefore assumes that the overall state has not changed.

Beispiel (Gemini) Stell dir einen Stack vor: $A \rightarrow B \rightarrow C$

Thread 1 will pop() machen: 1. Liest head = A (speichert lokal) 2. Liest next = B 3. Wird unterbrochen... (wird schlafen gelegt)

Thread 2 macht währenddessen: 1. pop(A) → Stack: B → C 2. pop(B) → Stack: C 3. push(A) → Stack: A → C (A ist wieder oben!)

Thread 1 wacht auf und macht CAS: - Prüft: "Ist head immer noch A?" → JA! - CAS gelingt → setzt top = B - Aber B existiert nicht mehr im Stack! Stack ist kaputt.

Remedies: - Pointer tagging (use bits as counter) - hazard pointers

Lock-Free Programming

Muster bei Lock Free: speicher geteilte variable in lokaler variable, am ende überprüfe ob sich geteilte variable geändert hat

```
do {
    head = top.get();           // 1. lies geteilte Variable lokal
    newi.next = head;          // 2. bereite Änderung vor
} while (!top.compareAndSet(head, newi)); // 3. war niemand dazwischen? → fertig
```

Stack mit Lock Free

```
public class ConcurrentStack {
    AtomicReference<Node> top = new AtomicReference<Node>();

    public void push(Long item) { ... }
    public Long pop() { ... }
}
```

```

public Long pop() {
    Node head, next;
    do {
        head = top.get();
        if (head == null) return null;
        next = head.next;
    } while (!top.compareAndSet(head, next));
    return head.item;
}

```

```

public void push(Long item) {
    Node newI = new Node(item);
    Node head;

    do {
        head = top.get();
        newI.next = head;
    } while (!top.compareAndSet(head, newI));
}

```

Performance

slow, weil viele Threads `top` wollen add exponential backoff (remember [04 Hardware Locks](#)) - threads fight for access to same resource slows down - solution: **go to sleep with random duration** → less try access at the same time - double waiting duration each time the resource is not free, reset when accessed

Linked list

Beim Stack reichte ein CAS. Bei einer verketteten Liste müssen beim Löschen zwei Dinge gleichzeitig passieren: 1. Den Node als "gelöscht" markieren 2. Den Zeiger des Vorgängers umhängen

Ohne Atomarität kann ein anderer Thread zwischen diesen zwei Schritten einfügen

`AtomicMarkableReference<V>`, pointer + flag bit. with `getReference()` we get the actual pointer, `attemptMark` lets us mark flag with boolean, `compareAndSet`.

Delete c:

1. try to set mark (c.next)
2. CAS([b.next.reference, b.next.marked], [c, unmarked], [d, unmarked])

Wenn Zeiger von b noch auf c zeigt UND b selbst nicht markiert ist ([c, unmarked]), dann ändere den Zeiger in einer unteilbaren Aktion so, dass er auf d zeigt UND unmarkiert bleibt ([d, unmarked])

Issue: when deleting 2 subsequent nodes, both get marked but only one gets actually removed with the pointer

Queue

- 2 pointer (head, tail)
- dummy always-in Placeholder node am Anfang "null" Fall zu verhindern

13 Execution Timelines

start: invocation end: return



Interactive visualization

<https://cs.rohlik.net/static/execution.html>

Quiescent consistency

1. non-overlapping operations have sequential effect
2. for overlapping operations we cannot say something (could even swap within thread)

Sequential consistency

"can we move actions left and right (without swapping) to make it work?"

aka Threads can interleave anyways, but within Thread order is set; we suppose write operation instantly visible

Linearizability

"can we find a point in time where each operation takes effect to make it work?"

Instead of a line where things happen, the change of state happens at a POINT between invocation and return. If there is a point for each action such that everything is correct, we say, it is linearizable.

- Linearisierbar $\implies$ Sequential consistency
- unlike in sequential consistency, we cannot move the operations (lines), but we decide on the point in time within the operation line

Describing the line situation formally: $\rightarrow_G \subset \rightarrow_S$: $\rightarrow_G$ G ist eine unvollständige Festlegung der absoluten Reihenfolge der Timeline, aber z.B. keine Aussage über overlapping Ops $\rightarrow_S$ G schränkt gibt restliche Reihenfolge die auch concurrent Ops eine Reihenfolge zuweist - H (history) ist linearisierbar, wenn es zu G erweitern kann indem man - appending responses to pending invocations that took effect (even though the program hasn't terminated but we see in other values that it has had effect already, so we set an ending-point) - discarding invocations that did not take effect (when no return no effect)

Korrektheits-Beweise mit Linearisierbarkeit, dann darf es nur einen Linearisierungspunkt pro Pfad geben.

 Interactive visualization

<https://cs.rohlik.net/static/consistency.html>

14 Consensus

Imaging 2 Threads propose 2 different numbers through a finite number of steps (wait free, independent), and they agree on a number and return.

1. **Valid** (the agreed number needs to be computed by either)
2. **Consistent** (both threads agree on same value)
3. **Wait-free** (one thread can return even though other thread kills, finite number of steps, no wait loop on other thread)

We implement by agreeing on first value (value of faster Thread).

Consensus Hierarchy

	Consensus number (how many threaded consensus can we create)
Atomic read/write registers	1
Test and Set	2
Compare and Set	n (beliebig)

Binary Consensus

a binary consensus is the equivalent to a normal consensus.

In the proof we use the ternary operator to convert a boolean to **1** and **0**, and vice versa.

Proof:

We can create a binary consensus when we have a normal consensus:

```
boolean binary_decide(boolean b) {  
    return int_decide(b ? 1 : 0) == 1;  
}
```

We can create a normal consensus when we have a binary consensus:

```
int int_decide(int d) {
    values[id] = d;
    int index = binary_decide(id == 1) ? 1 : 0;
    return values[index];
}
```

Consensus in the State Space Diagram

binary tree

- 2 children because 2 Threads
- no loop (because needs to be wait-free)
- each state contains: shared variables, local variables and program counter (the state contains both of A and B threads but Thread B doesn't see local variables and program counter of A), so from the perspective from one Thread the states look the same unless shared variables change

Begriffe

- **bivalent**: not yet decided if 0 or 1, both outcomes possible, start state is bivalent
- **univalent**: both children have same value, threads agreed already
- **critical**: diese Stelle ist gerade bivalent, aber entschieden was output ist, also ich kann nicht tiefer gehen und es ist immer noch nicht entschieden, bivalent with 2 univalent children

Lemma: Every consensus protocol has a critical state. *Proof:* In our State Space diagram we can only move down (no loop because wait-free), so at some point we reach an univalent state.

Java Implementation

2 Threaded

We use TAS, because this is not possible with volatile variables (see table above or proof below).

```
// Step 1: Announce own value to shared memory immediately
proposed[thread_id] = own_value;

// Step 2: Attempt to win the consensus race
if (TAS() == 0) {
    // I won. The decision is my value.
    return own_value;
} else {
    // I lost. The other thread won, so its value is the decision.
    // Read from shared memory AFTER losing.
    int other_thread_id = 1 - thread_id;
    return proposed[other_thread_id];
}
```

n-Threaded

We use CAS, not possible with TAS (see table above)

```
// single reference to one object
public class NThreadConsensus<T> {
    private final AtomicReference<T> decision = new AtomicReference<>(null);

    // every thread brings his proposal
    public T decide(T proposal) {
        // only for the first thread is decision empty, so he puts his proposal
        decision.compareAndSet(null, proposal);
        // we return the first threads' value
        return decision.get();
    }
}
```

Proof: We cannot do wait-free Consensus with only atomic registers

Proof We cover all options.

Assume we are at a critical state (there always is a critical state). Now, either Thread A or B needs to do something first. Wlog. we reduce to the following options for the next steps.

First action: - A: `r1.read()` - A: `r1.write()`

Second action: - B: `r1.read()` - B: `r2.read()` - B: `r1.write()` - B: `r2.write()`

If B is first, output 1, if A is first, output 0.

Case first action is A: `r1.read()` : - Thread A takes the first step, so our output needs to be 0. A pulls data to own private memory, but doesn't change shared memory. - Suppose, Thread A is now paused, Thread B continues itself (B can't wait for A because wait-free, so B must decide on its own). - But for Thread B the current situation (A read, now B's turn) is identical to as if B executed directly from the critical state, so it wouldn't know that it would need to output A's preference; Thread B would have to output 2 different values on the 2 branches. So now we have both A and B branches identical, thus the top node isn't critical $\implies$ contradiction. -

Case first action is A: `r1.write()` and second action is B `read()` - Same situation as above.

Case A and B write to different registers - Path left from critical node: - Thread A does `r1.write()`, so output must be 0 - Thread B does `r2.write()` - Path right from critical node: - Thread B does `r2.write()`, output must be 1 - Thread A does `r1.write()` - both paths lead to the same state, the registers have the same values, so no critical node

Case first action A: `r1.write()`, B: `r1.write()` - Same as Case 1, 2

15 Transactional Memory

Introduction

Problems with locks: - ordering is hard - can't combine thread safe operations - locks are pessimistic - synchronizations and locks are embedded in the code and not portable

Solution:

Transactional Memory

defined "atomic blocks" and not bother about the HOW

Ausführung passiert gleichzeitig, sieht für uns aber sequentiell aus.

Mögliche Ansätze - Thread erstellt lokale Kopie der Daten ("Snapshot"), macht die Änderungen dort, überprüft auf Konflikte und schreibt dann Änderungen atomar in den shared space, oder wiederholt - track changes, when a Thread makes a change to a value that is being used by another, abort early

Eigenschaften - TM ist atomar, aber nicht mutex - es können z.B. mehrere threads gleichzeitig beginnen - transactions appear serialized: es muss so aussehen können, als wäre die Ausführung sequentiell - Sobald eine Inkonsistenz zwischen lokaler Version und shared Version erkannt wird, wird abgebrochen (nicht einfach nicht committen) - TM can be implemented in Hardware or Software.

Strong vs weak isolation - **Strong isolation**: Garantiert Sicherheit, selbst wenn ein anderer Thread versucht, die Variable "normal" (ohne Transaktion, also ohne `atomic block`) zu lesen oder zu schreiben - **Weak isolation**: Shared state can **only** be read/modified in transactions (z.B. bei DiningPhilosophers muss ich auch bei put down forks `atomic block` haben)

Nesting Was passiert, wenn eine Transaktion innerhalb einer anderen gestartet wird? - **Flat Nesting**: im Grunde nur eine einzige, große Transaktion. Wenn die innere Transaktion scheitert, bricht die komplette (auch die äußere) Transaktion ab. - **Closed Nesting**: Die innere Transaktion kann abbrechen und es neu versuchen, ohne dass die äußere Transaktion abgebrochen werden muss.

ScalaSTM

- weak isolation
- closed nesting

Bank account example

Callable() when with return value instead of Runnable()

STM Retry: instead of aborting, retry: "wait bis es sich verändert", kein spin wait

Implementation of STM

- global clock (incremented with compare and set)
- each transaction reads latest value from clock, that's its "birthdate"
- when a transaction commits, the clock is incremented
- thread has "read set" (everything we have read from external) and "write set" (everything we have modified in local copy). each time check if needed Object is in write set already (local copy), if there, use. If not, get from external (check if timestamp of external value is younger as our transaction, if so, put into read set and continue. If not, abort.
- If all works out and we commit, we need to store into actual data. If it was only one object, easy (compare and set), but if multiple objects, needs locks (re-check everything again, then put elements from writeset to global including updated birthdate)

Message Passing

What if we avoid sharing state by using distributed memory? Each thread/task has its own private state, they cooperate via message passing.

Message Passing Interface for communication:

- a communicator is a set of threads that can talk about a task
- a thread can be part of multiple communicators
- each thread has a rank (id) within each communicator

How to **SPMD** (Single Program, Multiple Data)

```
if (rank == 0) {  
    // (Master-Prozess) Verteile die Arbeit  
} else {  
    // Du bist Worker-Prozess, rechne  
}
```

Send:

```
void Comm.Send(  
    Object buf, // the data array to be sent  
    int offset, // start index of relevant data  
    int count,  // number of elements relevant  
    Datatype datatype, // type of data  
    int dest,   // Rank (id) of destination thread/process  
    int tag     // id for this message  
)
```

Receive: equivalent function similar to `Comm.Send()`, but with `src` instead of `dest`

Synchronous Send: send, then wait for recipient to actively receive **Asynchronous Send:** send, then directly continue, buffer needed

Blocking: returns when buffer can be used again, but message transfer might not have been completed **Non-blocking:** return immediately